Padded mdspan layouts

Document #: P2642
Date: 2024-01-19
Project: Programming Language C++
LEWG
Reply-to: Christian Trott (Sandia National Laboratories)
<crtrott@sandia.gov>
Mark Hoemmen (NVIDIA)
<mhoemmen@nvidia.com>
Damien Lebrun-Grandie (Oak Ridge National Laboratory)
<lebrungrandt@ornl.gov>
Nicolas Morales (Sandia National Laboratories)
<nmmoral@sandia.gov>
Malte Förster (NVIDIA)
<mfoerster@nvidia.com>
Jiaming Yuan (NVIDIA)
<jiamingy@nvidia.com>

Contents

- 1 Authors
- 2 Revision history
  - 2.1 Revision 0
  - 2.2 Revision 1
  - 2.3 Revision 2
  - 2.4 Revision 3
  - 2.5 Revision 4
  - 2.6 Revision 5
  - 2.7 Revision 6
- 3 Proposed changes and justification
  - 3.1 Summary of proposed changes
  - 3.2 Two new mdspan layouts
    - 3.2.1 Optimizations over `layout_stride`
    - 3.2.2 New layouts unify two use cases
    - 3.2.3 Consider rank-1 case as `submdspan` of rank-2
    - 3.2.4 Design change from R0 to R1
    - 3.2.5 Padding stride equality for layout mapping conversions
    - 3.2.6 New layout mapping constructors in R2
    - 3.2.7 Conversion from `layout_left` to `layout_left_padded`
    - 3.2.8 Conversion from `layout_stride` to `layout_left_padded`
    - 3.2.9 Design change from R2 to R3: `extents()` return type
  - 3.3 Integration with `submdspan`
    - 3.3.1 `layout_left_padded` and `layout_left` cases
    - 3.3.2 `layout_right_padded` and `layout_right` cases
  - 3.4 Examples
    - 3.4.1 Directly call C BLAS without checks
    - 3.4.2 Overaligned access
  - 3.5 Design alternatives
    - 3.5.1 Strided layout with compile-time strides
    - 3.5.2 LEWG R2 polls discussion
    - 3.5.3 Nest the new policies in corresponding existing ones
    - 3.5.4 Layout mapping conversion customization point
  - 3.6 Implementation experience
  - 3.7 Desired ship vehicle
- 4 Wording
  - 4.1 Updates to *[mdspan.syn]*
  - 4.2 Updates to *[mdspan.layout.policy.overview]*
  - 4.3 Updates to *[mdspan.layout.left.overview]*
  - 4.4 Updates to *[mdspan.layout.left.cons]*
  - 4.5 Updates to *[mdspan.layout.right.overview]*
  - 4.6 Updates to *[mdspan.layout.right.cons]*
  - 4.7 Updates to *[mdspan.layout.general]*
  - 4.8 Updates to *[mdspan.layout.stride.cons]*
  - 4.9 Class template `layout_left_padded::mapping` [mdspan.layout.leftpadded]
    - 4.9.1 Overview [mdspan.layout.leftpadded.overview]
    - 4.9.2 Exposition-only members [mdspan.layout.leftpadded.expo]
    - 4.9.3 Constructors [mdspan.layout.leftpadded.cons]
    - 4.9.4 Observers [mdspan.layout.leftpadded.obs]
  - 4.10 Class template `layout_right_padded::mapping` [mdspan.layout.rightpadded]
    - 4.10.1 Overview [mdspan.layout.rightpadded.overview]
    - 4.10.2 Exposition-only members [mdspan.layout.rightpadded.expo]
    - 4.10.3 Constructors [mdspan.layout.rightpadded.cons]
    - 4.10.4 Observers [mdspan.layout.rightpadded.obs]
  - 4.11 Layout specializations of `submdspan_mapping` [mdspan.submdspan.mapping]

1 Authors

- Mark Hoemmen (mhoemmen@nvidia.com) (NVIDIA)
- Christian Trott (crtrott@sandia.gov) (Sandia National Laboratories)
- Damien Lebrun-Grandie (lebrungrandt@ornl.gov) (Oak Ridge National Laboratory)
- Malte Förster (mfoerster@nvidia.com) (NVIDIA)
- Jiaming Yuan (jiamingy@nvidia.com) (NVIDIA)

2 Revision history

2.1 Revision 0

Revision 0 submitted 2022-09-14.

2.2 Revision 1

Revision 1 submitted 2022-10-15.

- Change padding stride to function as an overalignment factor if less than the extent to pad. Remove mapping constructor that takes `extents_type` and `extents<index_type, padding_stride>`, because the latter may not be the actual padding stride. (NOTE: Revision 4 renamed `padding_stride` to `padding_value`.)
- Make converting constructors from `layout_{left,right}_padded::mapping` to `layout_{left,right}::mapping` use Mandates rather than Constraints to check compile-time stride compatibility.
- Mandate that `layout_{left,right}_padded::mapping`'s actual padding stride, if known at compile time, be representable as a value of type `index_type` (as well as of type `size_t`, the previous requirement).
- Add section explaining why we don't permit conversion from more aligned to less aligned.
- Fixed typos in Wording
- Fix formatting in non-Wording, and add links for BLAS and LAPACK

2.3 Revision 2

Revision 2 to be submitted 2023-01-15.

- Rebase atop P2630R2 (not R0).
- Fix synopsis declaration of `layout_{left,right}_padded` to declare the `mapping` as well
- Add that each specialization of `layout_{left,right}_padded` meets the layout mapping policy requirements and is a trivial type
- Simplify `layout_{left,right}_padded::mapping` constructor conditions
- Add default constructors for `layout_{left,right}_padded::mapping`; `=default` for *`static-padding-stride`*`!= dynamic_extent`, not `=default` otherwise
- Add converting constructors from `layout_(left,right)::mapping` to `layout_(left,right)_padded::mapping`
- Add converting constructors from `layout_stride::mapping` to `layout_(left,right)_padded::mapping`
- Add `operator==` to `layout_{left,right}_padded::mapping`
- In Remarks for the existing constructor `layout_stride::mapping(const StridedLayoutMapping&)`, add `layout_left_padded` and `layout_right_padded` to the list of the layouts in the expression inside `explicit`
- Reformat from Bikeshed to Pandoc

2.4 Revision 3

Revision 3 to be submitted sometime after 2023-07-09.

- Update P2630 (`submdspan`) revision number to R3.
- Add references to P2897 (`aligned_accessor`).
- Change return type of the padded layout mappings' `extents()` member functions from `extents_type` to `const extents_type&`, to make them consistent with the existing layout mapping requirements. As a result, change the padded layout mappings to include an exposition-only *`actual-extents`* member, so that `extents()` can return a valid reference. Thanks to Oliver Lee (oliverzlee@gmail.com) for an excellent discussion!
- Add design discussion requested by LEWG on 2023/03/28 relating to the results of the two polls.
- Add design discussion about `required_span_size()` of rank-1 padded layout mdspan.
- Update implementation experience with new pull request number.
- Change converting constructors from `layout_{left,right}_padded<ps>::mapping<OE>`, and `operator==` taking that type, to use a constraint instead of overloading. This helps with type deduction in practice. As a result, we needed a way to get the `padding_value` template argument of a mapping. We found it easiest to do this by adding a `static constexpr size_t padding_stride` public member to the mapping. Another approach would have been to introduce an exposition-only trait, but we think the public member would be more generally useful. (NOTE: Revision 4 renamed `padding_stride` to `padding_value`.)
- Add Nic Morales as a coauthor.

2.5 Revision 4

- bring wording in-line with layouts in the ISO C++ standard draft
- Rename `padding_stride` member of the two mappings to `padding_value`, to avoid confusion with the padding stride. Replace exposition-only *`actual-padding-stride`* with *`static-padding-stride`*.
- redo submdspan wording
  - in particular redo the specialization section for `submdspan_mapping`
  - split by layout type
  - pull common items for all layouts into a *Common* subsection

2.6 Revision 5

- Make change requested in 2023/11/09 LEWG poll: "Modify P2642R4 (Padded mdspan layouts) by removing the feature test macro and bumping the `__cpp_lib_submdspan` macro…."
- Remove "rebase on top of P2630R3" wording, since P2630 has been merged into the current C++ Working Draft. Update current C++ Working Draft reference to the latest at the time of publication, N4964.
- Update non-wording text and implementation experience.

2.7 Revision 6

- Remove inline definition of the two layout mappings' default constructors (the default constructor constrained with `static-padding-stride == dynamic_extent`), as the constructors were already defined in the text below.
- Fix update for [mdspan.layout.general].
- Fix constructor wording for initializing the stored extra stride, and the corresponding precondition.
- Respond to feedback from 2024/01/17 LWG review.
  - Change definitions of the exposition-only variable templates *`is-layout-left-padded-mapping-of`* and *`is-layout-right-padded-mapping-of`* to use prose instead of code.
  - Replace `size_t(0)` and other "literals cast to `size_t`" with actual integer literals (e.g., `0zu`).
  - Simplify *`static-padding-stride`* definition.
  - Simplify Mandates for `layout_left::mapping`'s converting constructor from `LayoutLeftPaddedMapping`, and for `layout_right::mapping`'s converting constructor from `LayoutRightPaddedMapping`. The "then" clause looks more restrictive, but is nevertheless equivalent to the previous wording.
  - Change the exposition-only `layout_left_padded<S>::mapping<E>::`*`stride-1`* and `layout_right_padded<S>::mapping<E>::`*`stride-rm2`* from having type `extents<index_type,`*`static-padding-stride`*`>` to having type `index_type`. For each of them, add a *Recommended practice* clause recommending not to store the value if *`static-padding-stride`* is not `dynamic_extent`.
  - Formatting fixes.
  - Generate padded layouts in more `submdspan` cases where possible.

3 Proposed changes and justification

3.1 Summary of proposed changes

We propose two new mdspan layouts, `layout_left_padded` and `layout_right_padded`. These layouts support two use cases:

1. array layouts that are contiguous in one dimension, as supported by commonly used libraries like the BLAS (Basic Linear Algebra Subroutines; see P1417, P1673, and P1674 for historical overview and references) and LAPACK (Linear Algebra PACKage); and
2. "padded" storage for overaligned access of the start of every contiguous segment of the array.

We also propose changing `submdspan` of a `layout_left` resp. `layout_right` mdspan to return `layout_left_padded` resp. `layout_right_padded` instead of `layout_stride`, when the slice arguments permit it.

3.2 Two new mdspan layouts

The two new mdspan layouts `layout_left_padded` and `layout_right_padded` are strided, unique layouts. If the rank is zero or one, then the layouts behave exactly like `layout_left` resp. `layout_right`. If the rank is two or more, then the layouts implement a special case of `layout_stride` where only one stride may differ from the extent that in `layout_left` resp. `layout_right` would completely define the stride. We call that stride the *padding stride*, and the extent that in `layout_left` resp. `layout_right` would define it the *extent to pad*. The padding stride of `layout_left_padded` is `stride(1)`, and the extent to pad is `extent(0)`. The padding stride of `layout_right_padded` is `stride(rank() - 2)`, and the extent to pad is `extent(rank() - 1)`. All other strides of `layout_left_padded` are the same as in `layout_left`, and all other strides of `layout_right_padded` are the same as in `layout_right`.

3.2.1 Optimizations over `layout_stride`

The two new layouts offer the following optimizations over `layout_stride`.

1. They guarantee at compile time that one extent always has stride-1 access. While `layout_stride`'s member functions are all `constexpr`, its mapping constructor takes the strides as a `std::array` with `rank()` size.
2. They do not need to store any strides if the padding stride is known at compile time. Even if the padding stride is a run-time value, these layouts only need to store the one stride value (as `index_type`). The `layout_stride::mapping` class must store all `rank()` stride values.

3.2.2 New layouts unify two use cases

The proposed layouts unify two different use cases:

1. overaligned access to the beginning of each contiguous segment of elements, and
2. representing exactly the data layout assumed by the General (GE) matrix type in the BLAS' C binding.

Regarding (1), an appropriate choice of padding can ensure any desired overalignment of the beginning of each contiguous segment of elements in an mdspan, as long as the entire memory allocation has the same overalignment. This is useful for hardware features that require or perform better with overaligned access, such as SIMD (Single Instruction Multiple Data) instructions.

Regarding (2), the padding stride is the same as BLAS' "leading dimension" of the matrix (`LDA`) argument. Unlike `layout_left` and `layout_right`, any subview of a contiguous subset of rows and columns of a rank-2 `layout_left_padded` or `layout_right_padded` mdspan preserves the layout. For example, if `A` is a rank-2 mdspan whose layout is `layout_left_padded<padding_value>`, then `submdspan(A, tuple{r1, r2}, tuple{c1, c2})` also has layout `layout_left_padded<padding_value>` with the same padding stride as before. The BLAS and algorithms that use it (such as the blocked algorithms in LAPACK) depend on this ability to operate on contiguous submatrices with the same layout as their parent. For this reason, we have replaced the `layout_blas_general` layout in earlier versions of our P1673 proposal with `layout_left_padded` and `layout_right_padded`. Making most effective use of the new layouts in code that uses P1673 calls for integrating them with `submdspan`. This is why we include `submdspan` integration in this proposal.

3.2.3 Consider rank-1 case as `submdspan` of rank-2

One review asked why `required_span_size()` of `layout_right_padded<4>::mapping<extents<size_t, 1, 3>>` is 4 instead of 3. We made that choice for the following reasons.

1. Overalignment should imply correct SIMD access as well as pointer alignment
2. Consistency of the rank-1 case with `submdspan` of a rank-2 mdspan

Regarding (1), an important design goal is use with explicit SIMD instructions. This means that we need to be able to access groups of 4 elements at a time. This is also consistent with `assume_aligned<N, T>`. That doesn't just return a pointer `p` such that `reinterpret_cast<uintptr_t>(p)` is divisible by `N`; it returns a pointer to an object of type `T` whose alignment is at least `N` bytes. `layout_right_padded<4>::mapping<extents<size_t, M, 3>>` for `M` in [1, 4] means "assume that each row is a `T[4]` with byte alignment `4 * sizeof(T)`."

Regarding (2), the idea is that the rank-2 or more case (with more than one row, column, etc.) controls the behavior of the rank-1 case. The rank-1 (or rank-2 but single row or column) case should act like a `submdspan` of the rank-2 case. It helps to understand that we intend to support the BLAS and LAPACK. For example, `layout_left_padded<4>::mapping<extents<size_t, 3, 3>>` means "a view of the top 3 rows of a 4 x 3 matrix" (LDA = 4, M = 3, N = 3, where LDA is a BLAS abbreviation meaning "leading dimension [of the matrix] A"). (This example switches to `layout_left_padded` just because the Fortran BLAS only supports column-major order, but the analogous idea applies to the `layout_right_padded` case that the C BLAS also supports.) Taking a `submdspan` of the leftmost column results in a rank-1 mdspan with a `required_span_size()` of 4 elements.

3.2.4 Design change from R0 to R1

A design change from R0 to R1 of this paper makes this overalignment case easier to use and more like the existing `std::assume_aligned` interface. In R0 of this paper, the user's padding input parameter (either a compile-time `padding_value` or a run-time value) was exactly the padding stride. As such, it had to be greater than or equal to the extent to pad. For example, if users had an `extent(0)` of 13 and wanted to overalign the corresponding `stride(1)` to a multiple of 4, they would have had to specify `layout_left_padded<16>`. This was inconsistent with `std::assume_aligned`, whose template argument (the byte alignment) would need to be `4 * sizeof(element_type)`. Also, users who wanted a compile-time padding stride would have needed to compute it themselves from the corresponding compile-time extent, rather than prespecifying a fixed overalignment factor that could be used for any extent. This was not only harder to use, but it made the layout itself (not just the layout mapping) depend on the extent. That was inconsistent with the existing mdspan layouts, where the layout type itself (e.g., `layout_left`) is always a function from `extents` specialization to layout mapping.

In R1 and subsequent revisions of this paper, we interpret the case where the input padding stride is less than the extent to pad as an "overalignment factor" instead of a stride. To revisit the above example, `layout_left_padded<4>` would take an `extent(0)` of 13 and round up the corresponding `stride(1)` to 16. However, as before, `layout_left_padded<17>` would take an `extent(0)` of 13 and round up the corresponding `stride(1)` to 17. The rule is consistent: the actual padding stride is always the next multiple of the input padding stride greater than or equal to the extent-to-pad.

In R0 of this paper, the following alias

```cpp
using overaligned_matrix_t =
  mdspan<float, dextents<size_t, 2>, layout_right_padded<4>>;
```

would only be meaningful if the run-time extents are less than or equal to 4. In R1 and subsequent revisions, this alias would always mean "the padding stride rounds up the rightmost extent to a multiple of 4, whatever the extent may be." R0 had no way to express that use case with a compile-time input padding stride. This is important for hardware features and compiler optimizations that require overalignment of multidimensional arrays.

3.2.5 Padding stride equality for layout mapping conversions

`layout_left_padded<padding_value>::mapping<Extents>` has a converting constructor from `layout_left_padded<other_padding_value>::mapping<OtherExtents>`. Similarly, `layout_right_padded<padding_value>::mapping<Extents>` has a converting constructor from `layout_right_padded<other_padding_value>::mapping<OtherExtents>`. These constructors require, among other conditions, that if `padding_value` and `other_padding_value` do not equal `dynamic_extent`, then `padding_value` equals `other_padding_value`.

Users may ask why they can't convert a more overaligned mapping, such as `layout_left_padded<4>::mapping`, to a less overaligned mapping, such as `layout_left_padded<2>::mapping`. The problem is that this may not be correct for all extents. For example, the following code would be incorrect if it were well formed (it is not, in this proposal).

```cpp
layout_left_padded<4>::mapping m_orig{extents{9, 2}};
layout_left_padded<2>::mapping m_new(m_orig);
```

The issue is that `m_orig` has an underlying ("physical") layout of `extents{12, 2}`, but `layout_left_padded<2>::mapping{extents{9, 2}}` would have an underlying layout of `extents{10, 2}`. That is, `layout_left_padded<4>::mapping{extents{9, 2}}.stride(1)` is 12, but `layout_left_padded<2>::mapping{extents{9, 2}}.stride(1)` is 10.

In case one is tempted to permit assigning dynamic padding stride to static padding stride, the following code would also be incorrect if it were well formed (it is not, in this proposal). Again, `m_orig.stride(1)` is 12.

```cpp
layout_left_padded<dynamic_extent>::mapping m_orig{extents{9, 2}, 4};
layout_left_padded<2>::mapping m_new(m_orig);
```

The following code is well formed in this proposal, and it gives `m_new` the expected original padding stride of 12.

```cpp
layout_left_padded<dynamic_extent>::mapping m_orig{extents{9, 2}, 4};
layout_left_padded<dynamic_extent>::mapping m_new(m_orig);
```

Similarly, the following code is well formed in this proposal, and it gives `m_new` the expected original padding stride of 12.

```cpp
layout_left_padded<4>::mapping m_orig{extents{9, 2}};
layout_left_padded<dynamic_extent>::mapping m_new(m_orig);
```

3.2.6 New layout mapping constructors in R2

R2 of this proposal adds new constructors to `layout_{left,right}_padded::mapping`. First, it adds default constructors that default-construct the `extents_type` object, but otherwise behave like the `mapping(const extents_type&)` constructor. That is, they fill in the correct run-time padding stride value, if this is possible given the `padding_value` template argument. Second, R2 adds more converting constructors. For `layout_left_padded::mapping`, R2 adds a converting constructor from each of the following.

- `layout_left::mapping<OtherExtents>`
- `layout_stride::mapping<OtherExtents>`

For `layout_right_padded::mapping`, R2 adds a converting constructor from each of the following.

- `layout_right::mapping<OtherExtents>`
- `layout_stride::mapping<OtherExtents>`

3.2.7 Conversion from `layout_left` to `layout_left_padded`

The converting constructor from `layout_left::mapping` to `layout_left_padded::mapping` exists by analogy with the existing constructor `layout_stride::mapping(const StridedLayoutMapping& other)` (*[mdspan.layout.stride.cons]*) that can convert from `layout_left::mapping` to `layout_stride::mapping`. `layout_left` expresses a special case of `layout_left_padded`, just as `layout_left` expresses a special case of `layout_stride`. Thus, this is an implicit conversion as long as the conversion from the input's `extents_type` to the result's `extents_type` would be implicit.

This conversion is useful for C++ wrappers for the BLAS or LAPACK. `layout_left_padded<dynamic_extent>::mapping<dextent<int, 2>>` expresses in C++ exactly the 2-D array layout that the BLAS and LAPACK accept, including their requirement that the extents and `stride(1)` all be run-time values. Thus, a C++ wrapper for the BLAS (see P1673) or LAPACK might reasonably have a specialization for `mdspan` with layout `layout_left_padded<dynamic_extent>::mapping<dextent<int, 2>>`, that can call with very few error checks or layout conversions directly into an existing C or Fortran BLAS or LAPACK library. However, users would reasonably want to create their 2-D arrays as `layout_left`, since it's a simpler layout that doesn't need to store the column stride. The converting constructor from `layout_left::mapping` to `layout_left_padded::mapping` would let users or libraries easily convert from the less general `layout_left` to the slightly more general `layout_left_padded` that a C++ BLAS or LAPACK wrapper would naturally use.

3.2.8 Conversion from `layout_stride` to `layout_left_padded`

The converting constructor from `layout_stride::mapping` to `layout_left_padded::mapping` exists by analogy with the existing converting constructor from `layout_stride::mapping` to `layout_left::mapping`. This constructor is `explicit` for `rank() > 0`, because it always converts from a more general case to a more specific case.

Explicit conversions to `layout_stride::mapping` are useful because `layout_stride::mapping` can express all the layout mappings in the Standard and this proposal. It's like a "type-erased" version of all of them. For example, a library of `mdspan` algorithms might reasonably convert to `layout_stride::mapping` for some less performance-critical algorithms, as a way to minimize algorithm instantiations for different layouts.

3.2.9 Design change from R2 to R3: `extents()` return type

In revisions of this proposal up to and including R2, the new layout mappings' `extents()` member functions both had return type `extents_type`. That is, they both returned by value. We did this deliberately, so that we could specify the layout mappings in terms of the behavior of `layout_left::mapping` resp. `layout_right::mapping` with a padded `extents` object, without needing to store the "original" `extents`. However, we realized after the publication of R2 that this does not respect the existing layout mapping requirements in paragraph 6 of *[mdspan.layout.reqmnts]*. This specifies the return type of `m.extents()` for every layout mapping as `const extents_type&`. That is, `extents()` must always return by const reference.

We considered changing the layout mapping requirements to permit layout mappings to return either `extents_type` or `const extents_type&`. However, we realized that *[mdspan.mdspan]* specifies that mdspan's `extent(r)` member function returns *`map_`*`.extents().extent(r)`. Letting a layout mapping's `extents()` create and return a temporary could make mdspan's `extent(r)` unexpectedly expensive. It should be always be cheap to get a single extent from an mdspan, because it's a common multidimensional array idiom to write nested `for` loops over each extent.

Our specification in *[mdspan.mdspan]* that mdspan's `extent(r)` returns *`map_`*`.extents().extent(r)` was also deliberate. It expresses two design choices. First, requiring mdspan to get its extents from its layout mapping (that is, specifying mdspan's `extents()` to return *`map_`*`.extents()`) ensures that an mdspan is nothing more that the composition of its data handle, layout mapping, and accessor. The layout mapping controls the extents; an mdspan cannot have "its own extents" that differ from those in its layout mapping. Second, not including `extents(r)` in the layout mapping means that a layout mapping also cannot have "its own extents" that differ from what `extents()` returns. Those two choices mean that the following code is well formed and does not trigger an `assert` for any mdspan `x`.

```cpp
// An mdspan's extents are its mapping's extents.
using mapping_type = decltype(x)::mapping_type;
using extents_type = mapping_type::extents_type;
static_assert(std::is_same_v<decltype(x)::extents_type, extents_type>);
assert(x.extents() == x.mapping().extents());

// A mapping's extent(r) must agree with its extents().
auto e = [&] <size_t... Indices> (std::index_sequence<Indices...>) {
    using index_type = extents_type::index_type;
    return extents<index_type, x.static_extent(Indices)...>{
      x.mapping().extent(Indices)...
    };
  } (std::make_index_sequence<x.rank()>());
static_assert(std::is_same_v<decltype(e), extents_type>);
assert(e == x.mapping().extents());
```

All these design choices add up to the padded layout mappings needing to return `const extents_type&` from `extents()`. This means that we cannot use R2's wording approach of having `extents()` return a temporary `extents` object. (Lifetime extension does not apply to a temporary created in and returned from a `return` statement.) Our wording fix in subsequent revisions is minimal: we add a new exposition-only *`actual-extents`* member of type `extents_type` to both of the padded mappings. However, this is not meant to suggest that implementations should take this approach. Instead of following the wording by using a nested `layout_left::mapping` resp. `layout_right::mapping` with a padded extents object, they could just reimplement the padded mappings as special cases of `layout_stride`. That way, each mapping would only store one `extents_type` object, and `extents()` would return a const reference to that object.

3.3 Integration with `submdspan`

We propose changing `submdspan` (see P2630, which was accepted into the C++ Working Draft for C++26) of a `layout_left` resp. `layout_right` mdspan to return `layout_left_padded` resp. `layout_right_padded` instead of `layout_stride`, if the slice arguments permit it. Taking the `submdspan` of a `layout_left_padded` resp. `layout_right_padded` mdspan will preserve the layout, again if the slice arguments permit it.

The phrase "if the slice arguments permit it" means the following.

3.3.1 `layout_left_padded` and `layout_left` cases

In what follows, let `left_submatrix` be the following function,

```cpp
template<class Elt, class Extents, class Layout,
  class Accessor, class S0, class S1>
requires(
  is_convertible_v<S0,
    tuple<typename Extents::index_type,
      typename Extents::index_type>> and
  is_convertible_v<S1,
    tuple<typename Extents::index_type,
      typename Extents::index_type>>
)
auto left_submatrix(
  mdspan<Elt, Extents, Layout, Accessor> X, S0 s0, S1 s1)
{
  auto full_extents =
    []<size_t ... Indices>(index_sequence<Indices...>) {
      return tuple{ (Indices, full_extent)... };
    }(make_index_sequence<X.rank() - 2>());
  return apply([&](full_extent_t ... fe) {
      return submdspan(X, s0, s1, fe...);
    }, full_extents);
}
```

let `index_type` be an integral type, let `s0` be an object of a type `S0` such that `is_convertible_v<S0, tuple<index_type, index_type>>` is `true`, and let `s1` be an object of a type `S1` such that `is_convertible_v<S1, tuple<index_type, index_type>>` is `true`.

Let `X` be an `mdspan` with rank at least two with `decltype(X)::index_type` naming the same type as `index_type`, whose layout is `layout_left_padded<padding_value_X>` for some `constexpr size_t padding_value_X`. Let `X_sub` be the object returned from `left_submatrix(X, s0, s1)`. Then, `X_sub` is an `mdspan` of rank `X.rank()` with layout `layout_left_padded<padding_value_X>`, and `X_sub.stride(1)` equals `X.stride(1)`.

Let `Z` be an `mdspan` with rank at least two with `decltype(Z)::index_type` naming the same type as `index_type`, whose layout is `layout_left`. Let `Z_sub` be the object returned from `left_submatrix(Z, s0, s1)`. Then, `Z_sub` is an `mdspan` of rank `Z.rank()` with layout `layout_left_padded<padding_value_Z>`, where `padding_value_Z` is

- `srm1_val1 - srm1_val0`, if `srm1` is convertible to `tuple<integral_constant<decltype(W)::index_type, srm1_val0>, integral_constant<decltype(W)::index_type, srm1_val1>>` with `srm1_val1` greater than to equal to `srm1_val0`; else,
- `dynamic_rank`.

Also, `Z_sub.stride(1)` equals `Z.stride(1)`.

3.3.2 `layout_right_padded` and `layout_right` cases

In what follows, let `right_submatrix` be the following function,

```cpp
template<class Elt, class Extents, class Layout,
  class Accessor, class Srm2, class Srm1>
requires(
  is_convertible_v<Srm2,
    tuple<typename Extents::index_type,
      typename Extents::index_type>> and
  is_convertible_v<Srm1,
    tuple<typename Extents::index_type,
      typename Extents::index_type>>
)
auto right_submatrix(
  mdspan<Elt, Extents, Layout, Accessor> X, Srm2 srm2, Srm1 srm1)
{
  auto full_extents =
    []<size_t ... Indices>(index_sequence<Indices...>) {
      return tuple{ (Indices, full_extent)... };
    }(make_index_sequence<X.rank() - 2>());
  return apply([&](full_extent_t ... fe) {
      return submdspan(X, fe..., srm2, srm1);
    }, full_extents);
}
```

let `srm2` ("s of rank minus 2") be an object of a type `Srm2` such that `is_convertible_v<S0, tuple<index_type_X, index_type_X>>` is `true`, and let `srm1` ("s of rank minus 1") be an object of a type `Srm1` such that `is_convertible_v<S1, tuple<index_type_X, index_type_X>>` is `true`.

Similarly, let `Y` be an `mdspan` with rank at least two whose layout is `layout_right_padded<padding_value_Y>` for some `constexpr size_t padding_value_Y`. Let `index_type_Y` name the type `decltype(Y)::index_type`. Let `srm2` ("S of rank minus 2") be an object of a type `Srm2` such that `is_convertible_v<Srm2, tuple<index_type_Y, index_type_Y>>` is `true`, and let `srm1` ("S of rank minus 1") be an object of a type `Srm1` such that `is_convertible_v<Srm1, tuple<index_type_Y, index_type_Y>>` is `true`. In the following code fragment,

```cpp
auto full_extents =
  []<size_t ... Indices>(index_sequence<Indices...>) {
    return tuple{(Indices, full_extent)...};
  }(make_index_sequence<Y.rank() - 2>());

auto Y_sub = apply([&](full_extent_t... fe) {
    return submdspan(Y, fe..., srm2, srm1);
  }, full_extents);
```

`Y_sub` is an `mdspan` of rank `Y.rank()` with layout `layout_left_padded<padding_value>`, and `Y_sub.stride(1)` equals `Y.stride(1)`.

Let `Z` be an `mdspan` with rank at least two whose layout is `layout_left`. Let `index_type_Z` name the type `decltype(Z)::index_type`. Let `s0` be an object of a type `S0` such that `is_convertible_v<S0, tuple<index_type_Z, index_type_Z>>` is `true`, and let `s1` be an object of a type `S1` such that `is_convertible_v<S1, tuple<index_type_Z, index_type_Z>>` is `true`. In the following code fragment,

```cpp
auto full_extents =
  []<size_t ... Indices>(index_sequence<Indices...>) {
    return tuple{(Indices, full_extent)...};
  }(make_index_sequence<Z.rank() - 2>());

auto Z_sub = apply( [&](full_extent_t... fe) {
    return submdspan(Z, s0, s1, fe...);
  }, full_extents );
```

`Z_sub` is an `mdspan` of rank `Z.rank()` with layout `layout_left_padded<padding_value_Z>`, where `padding_value_Z` is `s0_val1 - s0_val0` if `s0` is convertible to `tuple<integral_constant<index_type_Z, s0_val0>, integral_constant<index_type_Z, s0_val1>>` with `s0_val1` greater than to equal to `s0_val0`. Also, `Z_sub.stride(1)` equals `Z.stride(1)`.

Similarly, let `W` be an `mdspan` with rank at least two whose layout is `layout_right`. Let `index_type_W` name the type `decltype(W)::index_type`. Let `srm2` ("S of rank minus 2") be an object of a type `Srm2` such that `is_convertible_v<Srm2, tuple<index_type_W, index_type_W>>` is `true`, and let `srm1` ("S of rank minus 1") be an object of a type `Srm1` such that `is_convertible_v<Srm1, tuple<index_type_W, index_type_W>>` is `true`. In the following code fragment,

```cpp
auto full_extents =
  []<size_t ... Indices>(index_sequence<Indices...>) {
    return tuple{(Indices, full_extent)...};
  }(make_index_sequence<W.rank() - 2>());

auto W_sub = apply( [&](full_extent_t... fe) {
    return submdspan(W, fe..., srm2, srm1);
  }, full_extents);
```

`W_sub` is an `mdspan` of rank `W.rank()` with layout `layout_left_padded<padding_value_W>`, where `padding_value_W` is `srm1_val1 - srm1_val0` if `srm1` is convertible to `tuple<integral_constant<index_type_W, srm1_val0>, integral_constant<index_type_W, srm1_val1>>` with `srm1_val1` greater than to equal to `srm1_val0`. Also, `W_sub.stride(1)` equals `W.stride(1)`.

Preservation of these layouts under `submdspan` is an important feature for our linear algebra library proposal P1673 (which was accepted into the C++ Working Draft for C++26). It means that for existing BLAS and LAPACK use cases, if we start with one of these layouts, we know that we can implement fast linear algebra algorithms by calling directly into an optimized C or Fortran BLAS.

3.4 Examples

3.4.1 Directly call C BLAS without checks

We show examples before and after this proposal of functions that compute the matrix-matrix product $C += AB$. The `recursive_matrix_product` function computes this product recursively, by partitioning each of the three matrices into a 2 x 2 block matrix using the `partition` function. When the `C` matrix is small enough, `recursive_matrix_product` stops recursing and instead calls a `base_case_matrix_product` function with different overloads for different matrix layouts. If the matrix layouts support it, `base_case_matrix_product` can call the C BLAS function `cblas_sgemm` directly on the `mdspan`s' data. This is fast if the C BLAS is optimized. Otherwise, `base_case_matrix_product` falls back to a slow generic implementation.

This example is far from ideally optimized, but it hints at the kind of optimizations that linear algebra computations do in practice.

Common code:

```cpp
template<class Layout>
using out_matrix_view = mdspan<float, dextents<int, 2>, Layout>;

template<class Layout>
using in_matrix_view = mdspan<const float, dextents<int, 2>, Layout>;

// Before this proposal, if Layout is layout_left or layout_right,
// the returned mdspan would all be layout_stride.
// After this proposal, the returned mdspan would be
// layout_left_padded resp. layout_right_padded.
template<class ElementType, class Layout>
auto partition(mdspan<ElementType, dextents<int, 2>, Layout> A)
{
  auto M = A.extent(0);
  auto N = A.extent(1);
  auto A00 = submdspan(A, tuple{0, M / 2}, tuple{0, N / 2});
  auto A01 = submdspan(A, tuple{0, M / 2}, tuple{N / 2, N});
  auto A10 = submdspan(A, tuple{M / 2, M}, tuple{0, N / 2});
  auto A11 = submdspan(A, tuple{M / 2, M}, tuple{N / 2, N});
  return tuple{
    A00, A01,
    A10, A11
  };
}

template<class Layout>
void recursive_matrix_product(in_matrix_view<Layout> A,
  in_matrix_view<Layout> B, out_matrix_view<Layout> C)
{
  // Some hardware-dependent constant
  constexpr int recursion_threshold = 16;
  if(std::max(C.extent(0) || C.extent(1)) <= recursion_threshold) {
    base_case_matrix_product(A, B, C);
  } else {
    auto [C00, C01,
          C10, C11] = partition(C);
    auto [A00, A01,
          A10, A11] = partition(A);
    auto [B00, B01,
          B10, B11] = partition(B);
    recursive_matrix_product(A00, B00, C00);
    recursive_matrix_product(A01, B10, C00);
    recursive_matrix_product(A10, B00, C10);
    recursive_matrix_product(A11, B10, C10);
    recursive_matrix_product(A00, B01, C01);
    recursive_matrix_product(A01, B11, C01);
    recursive_matrix_product(A10, B01, C11);
    recursive_matrix_product(A11, B11, C11);
  }
}

// Slow generic implementation
template<class Layout>
void base_case_matrix_product(in_matrix_view<Layout> A,
  in_matrix_view<Layout> B, out_matrix_view<Layout> C)
{
  for(size_t j = 0; j < C.extent(1); ++j) {
    for(size_t i = 0; i < C.extent(0); ++i) {
      typename out_matrix_view<Layout>::value_type C_ij{};
      for(size_t k = 0; k < A.extent(1); ++k) {
        C_ij += A(i,k) * B(k,j);
      }
      C(i,j) += C_ij;
    }
  }
}
```

A user might interpret `layout_left` as "column major," and therefore "the natural layout to pass into the BLAS."

```cpp
void base_case_matrix_product(in_matrix_view<layout_left> A,
  in_matrix_view<layout_left> B, out_matrix_view<layout_left> C)
{
  cblas_sgemm(CblasColMajor, CblasNoTrans, CblasNoTrans,
    C.extent(0), C.extent(1), A.extent(1), 1.0f,
    A.data_handle(), A.stride(1), B.data_handle(), B.stride(1),
    1.0f, C.data_handle(), C.stride(1));
}
```

However, `recursive_matrix_product` never gets to use the `layout_left` overload of `base_case_matrix_product`, because the base case matrices are always `layout_stride`.

On discovering this, the author of these functions might be tempted to write a custom layout for "BLAS-compatible" matrices. However, `submdspan` as currently specified in the C++ Working Draft forces `partition` to return four `layout_stride` mdspan if given a `layout_left` (or `layout_right`) input mdspan. This would, in turn, force users of `recursive_matrix_product` to commit to a custom layout, if they want to use the BLAS.

Alternately, the author of these functions could specialize `base_case_matrix_product` for `layout_stride`, and check whether `A.stride(0)`, `B.stride(0)`, and `C.stride(0)` are all equal to one before calling `cblas_sgemm`. However, that would force extra run-time checks for a use case that most users might never encounter, because most users are starting with `layout_left` matrices or contiguous submatrices thereof.

After our proposal, the author can specialize `base_case_matrix_product` for exactly the layout supported by the BLAS. They could even get rid of the fall-back implementation if users never exercise it.

```cpp
template<size_t p>
void base_case_matrix_product(in_matrix_view<layout_left_padded<p>> A,
  in_matrix_view<layout_left_padded<p>> B,
  out_matrix_view<layout_left_padded<p>> C)
{ // same code as above
  cblas_sgemm(CblasColMajor, CblasNoTrans, CblasNoTrans,
    C.extent(0), C.extent(1), A.extent(1), 1.0f,
    A.data_handle(), A.stride(1), B.data_handle(), B.stride(1),
    1.0f, C.data_handle(), C.stride(1));
}
```

This optimization and simplification would also apply to implementations of P1673 that use a C or Fortran BLAS library where permitted by the `mdspan` layout(s) and accessor(s).

3.4.2 Overaligned access

By combining these new layouts with an accessor that ensures overaligned access, we can create an mdspan for which the beginning of every contiguous segment of elements is overaligned by some given factor. This can enable use of hardware features that require overaligned memory access.

The following `aligned_accessor` class template (proposed in our separate proposal P2897, which is currently in LEWG review as of the time of publication) uses the C++ Standard Library function `assume_aligned` to decorate pointer access.

```cpp
template<class ElementType, size_t byte_alignment>
struct aligned_accessor {
  using offset_policy = default_accessor<ElementType>;

  using element_type = ElementType;
  using reference = ElementType&;
  using data_handle_type = ElementType*;

  constexpr aligned_accessor() noexcept = default;

  template<class OtherElementType, size_t other_byte_alignment>
  requires (
    std::is_convertible_v<OtherElementType(*)[], element_type(*)[]> &&
    other_byte_alignment == byte_alignment)
  constexpr aligned_accessor(
    aligned_accessor<OtherElementType, other_byte_alignment>) noexcept
  {}
  constexpr reference
  access(data_handle_type p, size_t i) const noexcept {
    return std::assume_aligned< byte_alignment >(p)[i];
  }

  constexpr typename offset_policy::data_handle_type
  offset(data_handle_type p, size_t i) const noexcept {
    return p + i;
  }
};
```

We include some helper functions for making overaligned array allocations.

```cpp
template<class ElementType>
struct delete_raw {
  void operator()(ElementType* p) const {
    std::free(p);
  }
};

template<class ElementType>
using allocation_t =
  std::unique_ptr<ElementType[], delete_raw<ElementType>>;

template<class ElementType, std::size_t byte_alignment>
allocation_t<ElementType>
allocate_raw(const std::size_t num_elements)
{
  const std::size_t num_bytes = num_elements * sizeof(ElementType);
  void* ptr = std::aligned_alloc(byte_alignment, num_bytes);
  return {ptr, delete_raw<ElementType>{}};
}
```

Now we can show our example. This 15 x 17 matrix of `float` will have extra padding so that every column is aligned to `8 * sizeof(float)` bytes. We can use the layout mapping to determine the required storage size (including padding). Users can then prove at compile time that they can use special hardware features that require overaligned access and/or assume that the padding element at the end of each column is accessible memory.

```cpp
constexpr size_t element_alignment = 8;
constexpr size_t byte_alignment = element_alignment * sizeof(float);

using layout_type = layout_left_padded<element_alignment>;
layout_type::mapping mapping{dextents<int, 2>{15, 17}};
auto allocation =
  allocate_raw<float, byte_alignment>(mapping.required_span_size());

using accessor_type = aligned_accessor<float, byte_alignment>;
mdspan m{allocation.get(), mapping, accessor_type{}};

// m_sub has the same layout as m,
// and each column of m_sub has the same overalignment.
auto m_sub = submdspan(m, tuple{0, 11}, tuple{1, 13});
```

3.5 Design alternatives

3.5.1 Strided layout with compile-time strides

We considered a variant of `layout_stride` that could encode any combination of compile-time or run-time strides in the layout type. This could, for example, use the same mechanism that `extents` uses. (The reference implementation calls this mechanism a "partially static array.") However, we rejected this approach as overly complex for our design goals.

First, the goal of `layout_{left,right}_padded` isn't to insist even harder that the compiler bake constants into `mapping::operator()` evaluation. The goal is to communicate compile-time information to *users*. The most benefit comes not just from knowing the padding stride at compile time, but also from knowing that one dimension always uses stride-one (contiguous) storage. Putting these two pieces of information together lets users apply compiler annotations like `assume_aligned`, as in `aligned_accessor` (P2897). Knowing that one dimension always uses contiguous storage also tells users that they can pass the mdspan's data directly into C or Fortran libraries like the BLAS or LAPACK. Users can benefit from this even if the padding stride is a run-time value.

Second, the `constexpr` annotations in the existing layout mappings mean that users might be evaluating `layout_stride::mapping::operator()` fully at compile time. The reference mdspan implementation has several tests that demonstrate this by using the result of a layout mapping evaluation in a context where it needs to be known at compile time.

Third, the performance benefit of storing *some* strides as compile-time constants goes down as the rank increases, because most of the strides would end up depending on run-time values anyway. Strided mdspan generally come from a subview of an existing `layout_left` or `layout_right` mdspan. In that case, the representation of the strides that preserves the most compile-time information would be just the original mdspan's `extents_type` object. (Compare to the exposition-only *`inner-mapping`* which we use in the wording for `layout_{left,right}_padded`.) Computing each stride would then call for a forward (for `layout_left`) or reverse (for `layout_right`) product of the original mdspan's extents. As a result, any stride to the right resp. left of a run-time extent would end up depending on that run-time extent anyway. The larger the rank, the more strides get "touched" by run-time information.

Fourth, a strided mdspan that can represent layouts as general as `layout_stride`, but has entirely compile-time extents *and* strides, could be useful for supporting features of a specific computer architecture. However, these hardware features would probably have limitations that would prevent them from supporting general strided layouts anyway. For example, they might require strides to be a power of two, or they might be limited to specific ranges of extents or strides. These limitations would call for custom implementation-specific layouts, not something as general as a "compile-time `layout_stride`."

3.5.2 LEWG R2 polls discussion

LEWG's 2023 took two polls in its review of Revision 2 of this proposal on 2023/03/28. Both polls resulted in the status quo design, but LEWG asked us to add to the next revision a discussion of the questions they posed. We do so in the following sections.

3.5.3 Nest the new policies in corresponding existing ones

LEWG polled on the following question, with no votes in favor and thus no consensus for change. All coauthors present voted against.

> The proposed tagged type (`layout_left_padded`) should be a nested type (`layout_left::padded`).

The following will explain the context and why the authors oppose this change.

The suggestion was that we should change `layout_left_padded<padding_value>` and `layout_right_padded<padding_value>` from separate layout policies (the status quo) to nested types `layout_left::padded<padding_value>` resp. `layout_right::padded<padding_value>`.

The issue with this change is that it is "morphologically confused," to borrow the words of one LEWG reviewer. The layout mapping policy requirements *[mdspan.layout.policy.reqmts]* specify a shape (morphology) of two levels of types. The outer type `MP` is the layout mapping policy, which represents a family of layout mappings parameterized by `extents` type `E`. The inner type is the layout mapping `MP::mapping<E>`. Nesting a policy inside another policy, as in `layout_left::padded`, would break this rule that "the policy is on the outside, and the mapping is on the inside."

Note also that the `padding_value` template parameter must live outside the mapping. Otherwise, it wouldn't be possible to construct the mapping from just an `extents` object.

3.5.4 Layout mapping conversion customization point

LEWG polled on the following question, with no votes in favor and thus no consensus for change. One coauthor voted neutral and two coauthors voted weakly against.

> An `assume_layout` customization point should be provided for layout conversions.

The following will explain the context and why the authors oppose this change.

The status quo design includes converting constructors between some mappings, e.g., from `layout_left_padded::mapping<E1>` to `layout_left::mapping<E2>`. These constructors are conditionally `explicit` if there are nontrivial preconditions. This design matches the existing mdspan mapping conversions, e.g., from `layout_stride::mapping<E1>` to `layout_left::mapping<E2>`. The intent is that implicit conversions express and permit "type erasure," that is, going from information expressed in a compile-time type to information expressed in a member variable or some other way. Type erasure here includes three kinds of conversions.

1. From a more restrictive mapping to a less restrictive mapping (e.g., from `layout_left::mapping<E>` to `layout_stride::mapping<E>`)
2. From a mapping with static extents to a mapping with dynamic extents (e.g., from `layout_left::mapping<extents<int, 2, 3>>` to `layout_left::mapping<extents<int, dynamic_extent, 3>>`)
3. Both (changing the mapping and the extents)

Conversion in the opposite direction of type erasure generally imposes nontrivial preconditions, so it is explicit. We permit explicit conversions because they let users potentially improve performance by expressing their assumptions in the type system. If users didn't have explicit conversions, they would likely end up reimplementing them in a possibly less safe way.

LEWG reflector discussion suggested a more general approach to conversions. Instead of conditionally explicit constructors, conversions with nontrivial preconditions (e.g., from `layout_stride` to `layout_left_padded` to `layout_left`) would use an `assume_layout` customization point. This would have the following advantages.

1. Users could implement conversion from their custom layout mapping to a Standard layout mapping.
2. Adding a new layout to the Standard would not require adding converting constructors to all the existing mappings.

However, introducing a customization point would complicate the design. The benefit of this complication would be low, since most custom or new layout mappings likely could not be converted to existing layout mappings. For example, tiled layouts or space-filling (Hilbert) curve layouts are not strided in general, so they could not be converted to anything in the current Standard or this proposal. In the words of one LEWG reviewer, most layouts are "on [their] own little planet." LEWG discussion expressed a strong preference for not overengineering the design by offering a conversion customization point when most conversions don't make sense.

3.6 Implementation experience

The `stable` (main) branch of the reference mdspan implementation implements all of this proposal except `submdspan` support.

3.7 Desired ship vehicle

C++26 / IS.

4 Wording

> Text in blockquotes is not proposed wording, but rather instructions for generating proposed wording. The � character is used to denote a placeholder section number which the editor shall determine.
>
> Make the following changes to the latest C++ Working Draft, which at the time of writing is N4971. All wording is relative to the latest C++ Working Draft.
>
> In *[version.syn]*, increase the value of the `__cpp_lib_submdspan` macro by replacing YYYMML below with the integer literal encoding the appropriate year (YYYY) and month (MM).

```cpp
#define __cpp_lib_submdspan YYYYMML // also in <mdspan>
```

4.1 Updates to *[mdspan.syn]*

> In Section � *[mdspan.syn]*, in the synopsis, after `struct layout_stride;`, add the following:

```cpp
// [mdspan.layout], layout mapping
struct layout_left;
struct layout_right;
struct layout_stride;
template<size_t PaddingValue = dynamic_extent>
struct layout_left_padded;
template<size_t PaddingValue = dynamic_extent>
struct layout_right_padded;
```

4.2 Updates to *[mdspan.layout.policy.overview]*

> In *[mdspan.layout.policy.overview]*, add the following to the code block after the `layout_stride` definition:

```cpp
template<size_t PaddingValue>
struct layout_left_padded {
  template<class Extents>
  class mapping;
};
template<size_t PaddingValue>
struct layout_right_padded {
  template<class Extents>
  class mapping;
};
```

> After paragraph 1 of *[mdspan.layout.policy.overview]*, add the following paragraph 2:

2 Each specialization of `layout_left_padded` and `layout_right_padded` meets the layout mapping policy requirements and is a trivial type.

4.3 Updates to *[mdspan.layout.left.overview]*

> In Section � *[mdspan.layout.left.overview]* ("Overview"), add the following constructor to the `layout_left::mapping` class declaration, between the constructor converting from `layout_right::mapping<OtherExtents>` and the constructor converting from `layout_stride::mapping<OtherExtents>`:

```cpp
template<class LayoutLeftPaddedMapping>
  constexpr explicit(! is_convertible_v<typename LayoutLeftPaddedMapping::extents_type, extents_type>)
    mapping(const LayoutLeftPaddedMapping&) noexcept;
```

4.4 Updates to *[mdspan.layout.left.cons]*

> In Section � *[mdspan.layout.left.cons]* ("Constructors"), add the following between the constructor converting from `layout_right::mapping<OtherExtents>` (ending paragraph 8) and the constructor converting from `layout_stride::mapping<OtherExtents>` (starting paragraph 9 before this proposal), then renumber the following paragraphs in that section accordingly.

```cpp
template<class LayoutLeftPaddedMapping>
  constexpr explicit(! is_convertible_v<typename LayoutLeftPaddedMapping::extents_type, extents_type>)
    mapping(const LayoutLeftPaddedMapping& other) noexcept;
```

9 *Constraints:*

(9.1) — *`is-layout-left-padded-mapping-of`*`<LayoutLeftPaddedMapping>` is `true`.

(9.2) — `is_constructible_v<extents_type, typename LayoutLeftPaddedMapping::extents_type>` is `true`.

10 *Mandates:* If

- `Extents::rank()` is greater than one,
- `Extents::static_extent(0)` does not equal `dynamic_extent`, and
- `LayoutLeftPaddedMapping::`*`static-padding-stride`* does not equal `dynamic_extent`,

then `Extents::static_extent(0)` equals `LayoutLeftPaddedMapping::`*`static-padding-stride`*.

11 *Preconditions:*

(11.1) — If `extents_type::rank() > 1` is `true`, then `other.stride(1)` equals `other.extents(0)`.

(11.2) — `other.required_span_size()` is representable as a value of type `index_type`.

12 *Effects:* Direct-non-list-initializes `extents_` with `other.extents()`.

4.5 Updates to *[mdspan.layout.right.overview]*

> In Section � *[mdspan.layout.right.overview]* ("Overview"), add the following constructor to the `layout_right::mapping` class declaration, between the constructor converting from `layout_left::mapping<OtherExtents>` and the constructor converting from `layout_stride::mapping<OtherExtents>`.

```cpp
template<class LayoutRightPaddedMapping>
  constexpr explicit(! is_convertible_v<typename LayoutRightPaddedMapping::extents_type, extents_type>)
    mapping(const LayoutRightPaddedMapping&) noexcept;
```

4.6 Updates to *[mdspan.layout.right.cons]*

> In Section � *[mdspan.layout.right.cons]* ("Constructors"), add the following between the constructor converting from `layout_left::mapping<OtherExtents>` (ending paragraph 8) and the constructor converting from `layout_stride::mapping<OtherExtents>` (starting paragraph 9 before this proposal), then renumber the following paragraphs in that section accordingly.

```cpp
template<class LayoutRightPaddedMapping>
  constexpr explicit(! is_convertible_v<typename LayoutRightPaddedMapping::extents_type, extents_type>)
    mapping(const LayoutRightPaddedMapping& other) noexcept;
```

9 *Constraints:*

(9.1) — *`is-layout-right-padded-mapping-of`*`<LayoutRightPaddedMapping>` is `true`.

(9.2) — `is_constructible_v<extents_type, typename LayoutRightPaddedMapping::extents_type>` is `true`.

10 *Mandates:* If

- `Extents::rank()` is greater than one,
- `Extents::static_extent(Extents::rank() - 1)` does not equal `dynamic_extent`, and
- `LayoutRightPaddedMapping::`*`static-padding-stride`* does not equal `dynamic_extent`,

then `Extents::static_extent(Extents::rank() - 1)` equals `LayoutRightPaddedMapping::`*`static-padding-stride`*.

11 *Preconditions:*

(11.1) — if `extents_type::rank() > 1` is `true`, then `other.stride(extents_type::rank() - 2)` equals `other.extents().extent(extents_type::rank() - 1)`.

(11.2) — `other.required_span_size()` is representable as a value of type `index_type`.

12 *Effects:* Direct-non-list-initializes `extents_` with `other.extents()`.

4.7 Updates to *[mdspan.layout.general]*

> In Section � *[mdspan.layout.general]*, change paragraph 2 to be:

2 In subclauses *[mdspan.layout.reqmts]* through *[mdspan.layout.rightpadded]*:

(2.1) — Let *`is-mapping-of`* be the exposition-only variable template defined as follows:

```cpp
template<class Layout, class Mapping>
constexpr bool is-mapping-of = // exposition only
  is_same_v<typename Layout::template mapping<typename Mapping::extents_type>, Mapping>;
```

(2.2) — Let *`is-layout-left-padded-mapping-of`* be the exposition-only variable template defined as follows:

```cpp
template<class Mapping>
constexpr bool is-layout-left-padded-mapping-of = // exposition only
  /* see-below */;
```

where *`is-layout-left-padded-mapping-of`*`<Mapping>` is `true` if and only if `Mapping` denotes a specialization of `layout_left_padded<S>::mapping` for some value `S` of type `size_t`.

(2.3) — Let *`is-layout-right-padded-mapping-of`* be the exposition-only variable template defined as follows:

```cpp
template<class Mapping>
constexpr bool is-layout-right-padded-mapping-of = // exposition only
  /* see-below */;
```

where *`is-layout-right-padded-mapping-of`*`<Mapping>` is `true` if and only if `Mapping` denotes a specialization of `layout_right_padded<S>::mapping` for some value `S` of type `size_t`.

(2.4) — For nonnegative integers x and y, let *LEAST-MULTIPLE-AT-LEAST*(x, y) denote

- y if x is zero, otherwise
- the least multiple of x that is greater than or equal to y.

4.8 Updates to *[mdspan.layout.stride.cons]*

> In Section � *[mdspan.layout.stride.cons]*, for the following constructor whose description starts on paragraph 6,

```cpp
template<class StridedLayoutMapping>
  constexpr explicit(see below)
    mapping(const StridedLayoutMapping& other) noexcept;
```

> change paragraph 9 (*Remarks*) by adding the two lines shown in green to the code expression that follows the "is equivalent to" phrase.

9 *Remarks*: The expression inside `explicit` is equivalent to:

```cpp
! (is_convertible_v<typename StridedLayoutMapping::extents_type, extents_type> &&
(is-mapping-of<layout_left, StridedLayoutMapping> ||
is-mapping-of<layout_right, StridedLayoutMapping> ||
is-layout-left-padded-mapping-of<StridedLayoutMapping> ||
is-layout-right-padded-mapping-of<StridedLayoutMapping> ||
is-mapping-of<layout_stride, StridedLayoutMapping>))
```

> After the end of Section � *[mdspan.layout.stride]*, add the following:

4.9 Class template `layout_left_padded::mapping` [mdspan.layout.leftpadded]

4.9.1 Overview [mdspan.layout.leftpadded.overview]

1 `layout_left_padded` provides a layout mapping that behaves like `layout_left::mapping`, except that the *padding stride* `stride(1)` can be greater than or equal to `extent(0)`.

```cpp
template<size_t PaddingValue>
template<class Extents>
class layout_left_padded<PaddingValue>::mapping {
public:
  static constexpr size_t padding_value = PaddingValue;

  using extents_type = Extents;
  using index_type = typename extents_type::index_type;
  using size_type = typename extents_type::size_type;
  using rank_type = typename extents_type::rank_type;
  using layout_type = layout_left_padded<PaddingValue>;

private:
  static constexpr size_t rank_ = extents_type::rank(); // exposition only
  static constexpr size_t first-static-extent = // exposition only
    extents_type::static_extent(0);

  // [mdspan.layout.leftpadded.expo], exposition-only members
  static constexpr size_t static-padding-stride = /* see-below */; // exposition only

public:
  // [mdspan.layout.leftpadded.cons], constructors
  constexpr mapping() noexcept : mapping(extents_type{}) {}
  constexpr mapping(const mapping&) noexcept = default;
  constexpr mapping(const extents_type&);
  template<class OtherIndexType>
    constexpr mapping(const extents_type&, OtherIndexType);

  template<class OtherExtents>
    constexpr explicit(! is_convertible_v<OtherExtents, extents_type>)
      mapping(const layout_left::mapping<OtherExtents>&);
  template<class OtherExtents>
    constexpr explicit(extents_type::rank() > 0)
      mapping(const layout_stride::mapping<OtherExtents>&);
  template<class LayoutLeftPaddedMapping>
    constexpr explicit( /* see below */ )
      mapping(const LayoutLeftPaddedMapping&);
  template<class LayoutRightPaddedMapping>
    constexpr explicit( /* see below */ )
      mapping(const LayoutRightPaddedMapping&) noexcept;

  constexpr mapping& operator=(const mapping&) noexcept = default;

  // [mdspan.layout.leftpadded.obs], observers
  constexpr const extents_type& extents() const noexcept { return extents_; }
  constexpr array<index_type, rank_> strides() const noexcept;

  constexpr index_type required_span_size() const noexcept;

  template<class... Indices>
    constexpr index_type operator()(Indices...) const noexcept;

  static constexpr bool is_always_unique() noexcept { return true; }
  static constexpr bool is_always_exhaustive() noexcept;
  static constexpr bool is_always_strided() noexcept { return true; }

  static constexpr bool is_unique() noexcept { return true; }
  constexpr bool is_exhaustive() const noexcept;
  static constexpr bool is_strided() noexcept { return true; }

  constexpr index_type stride(rank_type) const noexcept;

  template<class LayoutLeftPaddedMapping>
    friend constexpr bool operator==(
      const mapping&,
      const LayoutLeftPaddedMapping&) noexcept;

private:
  // [mdspan.layout.leftpadded.expo], exposition-only members
  index_type stride-1 = static-padding-stride; // exposition only
  extents_type extents_{}; // exposition only

  // [mdspan.submdspan.mapping], submdspan mapping specialization
  template<class... SliceSpecifiers>
    constexpr auto submdspan-mapping-impl( // exposition only
      SliceSpecifiers...) const -> see below;

  template<class... SliceSpecifiers>
    friend constexpr auto submdspan_mapping(
      const mapping& src, SliceSpecifiers... slices) {
        return src.submdspan-mapping-impl(slices...);
  }
};
```

2 If `Extents` is not a specialization of `extents`, then the program is ill-formed.

3 `layout_left_padded::mapping<E>` is a trivially copyable type that models `regular` for each `E`.

4 Throughout [mdspan.layout.leftpadded], let `P_rank` be the following size *`rank_`* parameter pack of `size_t` values:

(4.1) — the empty parameter pack, if *`rank_`* equals zero; otherwise

(4.2) — `0zu`, if *`rank_`* equals one; otherwise

(4.3) — the parameter pack `0zu`, `1zu`, …, *`rank_`*`- 1`.

5 *Mandates:*

(5.1) — If `rank_dynamic() == 0` is `true`, then the size of the multidimensional index space `Extents()` is representable as a value of type `index_type`.

(5.2) — `padding_value` is representable as a value of type `index_type`.

(5.3) — If

- *`rank_`* is greater than one,
- `padding_value` does not equal `dynamic_extent`, and
- *`first-static-extent`* does not equal `dynamic_extent`,

then *LEAST-MULTIPLE-AT-LEAST*(`padding_value` , *`first-static-extent`*) is representable as a value of type `size_t`, and is representable as a value of type `index_type`.

(5.4) — If

- *`rank_`* is greater than one,
- `padding_value` does not equal `dynamic_extent`, and
- `extents_type::static_extent(k)` does not equal `dynamic_extent` for all `k` in the range [0, `extents_type::rank()`),

then the product of *LEAST-MULTIPLE-AT-LEAST*(`padding_value` , `ext.static_extent(0)`) and all values `ext.static_extent(k)` with `k` in the range of [1, *`rank_`*) is representable as a value of type `size_t`, and is representable as a value of type `index_type`.

4.9.2 Exposition-only members [mdspan.layout.leftpadded.expo]

```cpp
static constexpr size_t static-padding-stride = /* see-below */; // exposition only
```

1 The value is

(1.1) — `0`, if *`rank_`* equals zero or one; otherwise

(1.2) — `dynamic_extent`, if `padding_value` or *`first-static-extent`* equals `dynamic_extent`; otherwise

(1.3) — the `size_t` value which is *LEAST-MULTIPLE-AT-LEAST*(`padding_value` , *`first-static-extent`*).

```cpp
index_type stride-1 = static-padding-stride; // exposition only
```

2 *Recommended practice*: Implementations should not store this value if *`static-padding-stride`* is not `dynamic_extent`. *[Note:* Using `extents<index_type,`*`static-padding-stride`*`>` instead of `index_type` as the type of *`stride-1`* would achieve this. *– end note]*

4.9.3 Constructors [mdspan.layout.leftpadded.cons]

```cpp
constexpr mapping(const extents_type& ext);
```

1 *Preconditions:*

(1.1) — The size of the multidimensional index space `ext` is representable as a value of type `index_type`.

(1.2) — If *`rank_`* is greater than one and `padding_value` does not equal `dynamic_extent`, then *LEAST-MULTIPLE-AT-LEAST*(`padding_value` , `ext.extent(0)`) is representable as a value of type `index_type`.

(1.3) — If *`rank_`* is greater than one and `padding_value` does not equal `dynamic_extent`, then the product of *LEAST-MULTIPLE-AT-LEAST*(`padding_value` , `ext.extent(0)`) and all values `ext.extent(k)` with `k` in the range of [1, *`rank_`*) is representable as a value of type `index_type`.

2 *Effects:*

(2.1) — Direct-non-list-initializes *`extents_`* with `ext`; and

(2.2) — if *`rank_`* is greater than one, direct-non-list-initializes *`stride-1`* with

(2.2.1) — `ext.extent(0)` if `padding_value` is `dynamic_extent`, otherwise with

(2.2.2) — *LEAST-MULTIPLE-AT-LEAST*(`padding_value` , `ext.extent(0)`).

```cpp
template<class OtherIndexType>
constexpr mapping(const extents_type& ext, OtherIndexType pad);
```

3 *Constraints:*

(3.1) — `is_convertible_v<OtherIndexType, index_type>` is `true`.

(3.2) — `is_nothrow_constructible_v<index_type, OtherIndexType>` is `true`.

4 *Preconditions:*

(4.1) — `pad` is representable as a value of type `index_type`.

(4.2) — `extents_type::`*`index-cast`*`(pad)` is greater than zero.

(4.3) — If *`rank_`* is greater than one, then *LEAST-MULTIPLE-AT-LEAST*(`pad` , `ext.extent(0)`) is representable as a value of type `index_type`.

(4.4) — If *`rank_`* is greater than one, then the product of *LEAST-MULTIPLE-AT-LEAST*(`pad` , `ext.extent(0)`) and all values `ext.extent(k)` with `k` in the range of [1, *`rank_`*) is representable as a value of type `index_type`.

(4.5) — If `padding_value` is not equal to `dynamic_extent`, `padding_value` equals `extents_type::`*`index-cast`*`(pad)`.

5 *Effects:* Direct-non-list-initializes *`extents_`* with `ext`, and if *`rank_`* is greater than one, direct-non-list-initializes *`stride-1`* with *LEAST-MULTIPLE-AT-LEAST*(`pad` , `ext.extent(0)`).

```cpp
template<class OtherExtents>
  constexpr explicit(! is_convertible_v<OtherExtents, extents_type>)
    mapping(const layout_left::mapping<OtherExtents>& other);
```

6 *Constraints:* `is_constructible_v<extents_type, OtherExtents>` is `true`.

7 *Mandates:* If `OtherExtents::rank()` is greater than 1, then `(`*`static-padding-stride`*`== dynamic_extent) || (OtherExtents::static_extent(0) == dynamic_extent) || (`*`static-padding-stride`*`== OtherExtents::static_extent(0))` is `true`.

8 *Preconditions:*

(8.1) — If `extents_type::rank() > 1` is `true` and `padding_value == dynamic_extent` is `false`, then `other.stride(1)` equals *LEAST-MULTIPLE-AT-LEAST*(`padding_value` , `extents_type::`*`index-cast`*`(other.extents().extent(0))`); and

(8.2) — `other.required_span_size()` is representable as a value of type `index_type`.

9 *Effects:* Equivalent to `mapping(other.extents())`.

```cpp
template<class OtherExtents>
  constexpr explicit(rank_ > 0)
    mapping(const layout_stride::mapping<OtherExtents>& other);
```

10 *Constraints:* `is_constructible_v<extents_type, OtherExtents>` is `true`.

11 *Preconditions:*

(11.1) — If *`rank_`* is greater than 1 and `padding_value` does not equal `dynamic_extent`, then `other.stride(1)` equals *LEAST-MULTIPLE-AT-LEAST*(`padding_value` , `extents_type::`*`index-cast`*`(other.extents().extent(0))`).

(11.2) — If *`rank_`* is greater than 0, then `other.stride(0)` equals 1.

(11.3) — If *`rank_`* is greater than 2, then for all `r` in the range [2, *`rank_`*), `other.stride(r)` equals `(other.extents().`*`fwd-prod-of-extents`*`(r) / other.extents().extent(0)) * other.stride(1)`.

(11.4) — `other.required_span_size()` is representable as a value of type `index_type`.

12 *Effects:*

(12.1) — Direct-non-list-initializes *`extents_`* with `other.extents()`; and

(12.2) — if *`rank_`* is greater than one, direct-non-list-initializes *`stride-1`* with `other.stride(1)`.

```cpp
template<class LayoutLeftPaddedMapping>
  constexpr explicit( /* see below */ )
    mapping(const LayoutLeftPaddedMapping& other);
```

13 *Constraints:*

(13.1) — *`is-layout-left-padded-mapping-of`*`<LayoutLeftPaddedMapping>` is `true`.

(13.2) — `is_constructible_v<extents_type, typename LayoutLeftPaddedMapping::extents_type>` is `true`.

14 *Mandates:* If *`rank_`* is greater than 1, then `padding_value == dynamic_extent || LayoutLeftPaddedMapping::padding_value == dynamic_extent || padding_value == LayoutLeftPaddedMapping::padding_value` is `true`.

15 *Preconditions:*

(15.1) — If *`rank_`* is greater than 1 and `padding_value` does not equal `dynamic_extent`, then `other.stride(1)` equals *LEAST-MULTIPLE-AT-LEAST*(`padding_value` , `extents_type::`*`index-cast`*`(other.extent(0))`).

(15.2) — `other.required_span_size()` is representable as a value of type `index_type`.

16 *Effects:*

(16.1) — Direct-non-list-initializes *`extents_`* with `other.extents()`; and

(16.2) — if *`rank_`* is greater than one, direct-non-list-initializes *`stride-1`* with `other.stride(1)`.

17 *Remarks:* The expression inside `explicit` is equivalent to: *`rank_`*`> 1 && (padding_value != dynamic_extent || LayoutLeftPaddedMapping::padding_value == dynamic_extent)`.

```cpp
template<class LayoutRightPaddedMapping>
  constexpr explicit( /* see below */ )
    mapping(const LayoutRightPaddedMapping& other) noexcept;
```

18 *Constraints:*

(18.1) — *`is-layout-right-padded-mapping-of`*`<LayoutRightPaddedMapping>` is `true` or *`is-mapping-of`*`<layout_right, LayoutRightPaddedMapping>` is `true`.

(18.2) — *`rank_`* equals zero or one.

(18.3) — `is_constructible_v<extents_type, typename LayoutRightPaddedMapping::extents_type>` is `true`.

19 *Precondition:* `other.required_span_size()` is representable as a value of type `index_type`.

20 *Effects:* direct-non-list-initializes *`extents_`* with `other.extents()`.

21 *Remarks:* The expression inside `explicit` is equivalent to: `! is_convertible_v<typename LayoutRightPaddedMapping::extents_type, extents_type>`.

[Note: Neither the input mapping nor the mapping to be constructed uses the padding stride in the rank-0 or rank-1 case, so the padding stride does not affect either the constraints or the preconditions. *– end note]*

4.9.4 Observers [mdspan.layout.leftpadded.obs]

```cpp
constexpr array<index_type, rank_>
  strides() const noexcept;
```

1 *Returns:* `array<index_type,`*`rank_`*`>({stride(P_rank)...})`.

```cpp
constexpr index_type required_span_size() const noexcept;
```

2 *Returns:*

(2.1) — `0` if the multidimensional index space *`extents_`* is empty, otherwise

(2.2) — `*this(((extents_(P_rank) - index_type(1))...)) + 1`.

```cpp
template<class... Indices>
constexpr size_t operator()(Indices... idxs) const noexcept;
```

3 *Constraints:*

(3.1) — `sizeof...(Indices) ==`*`rank_`* is `true`.

(3.2) — `(is_convertible_v<Indices, index_type> && ...)` is `true`.

(3.3) — `(is_nothrow_constructible_v<index_type, Indices> && ...)` is `true`.

4 *Precondition:* `extents_type::`*`index-cast`*`(idxs)` is a multidimensional index in `extents()` (*[mdspan.overview]*).

5 *Returns:* `((static_cast<index_type>(idxs) * stride(P_rank)) + ... + 0)`.

```cpp
static constexpr bool is_always_exhaustive() noexcept;
```

6 *Returns:*

(6.1) — If *`rank_`* equals zero or one, then `true`;

(6.2) — else, if neither *`static-padding-stride`* nor *`first-static-extent`* equal `dynamic_extent`, then *`static-padding-stride`*`==`*`first-static-extent`*;

(6.3) — otherwise, `false`.

```cpp
constexpr bool is_exhaustive() const noexcept;
```

7 *Returns:* `true` if *`rank_`* equals zero or one; otherwise, *`extents_`*`.extent(0) == stride(1)`.

```cpp
constexpr index_type stride(rank_type r) const noexcept;
```

8 *Preconditions:* `r` is smaller than *`rank_`*.

9 *Returns:*

(9.1) — If `r` equals zero: `1`;

(9.2) — else, if `r` equals one: *`stride-1`*;

(9.3) — otherwise, the product of *`stride-1`* and all values *`extents_`*`.extent(k)` with `k` in the range of [1, `r`).

```cpp
template<class LayoutLeftPaddedMapping>
  friend constexpr bool operator==(
    const mapping& x,
    const LayoutLeftPaddedMapping& y) noexcept;
```

10 *Constraints:*

(10.1) — *`is-layout-left-padded-mapping-of`*`<LayoutLeftPaddedMapping>` is `true`.

(10.2) — `LayoutLeftPaddedMapping::extents_type::rank() ==`*`rank_`* is `true`.

11 *Returns:* `true` if `x.extents() == y.extents()` is `true` and *`rank_`*`< 2 || x.stride(1) == y.stride(1)` is `true`. Otherwise, `false`.

4.10 Class template `layout_right_padded::mapping` [mdspan.layout.rightpadded]

4.10.1 Overview [mdspan.layout.rightpadded.overview]

1 `layout_right_padded` provides a layout mapping that behaves like `layout_right::mapping`, except that the *padding stride* `stride(extents_type::rank()-2)` can be greater than or equal to `extents_type::extent(extents_type::rank()-1)`.

```cpp
template<size_t PaddingValue>
template<class Extents>
class layout_right_padded<PaddingValue>::mapping {
public:
  static constexpr size_t padding_value = PaddingValue;

  using extents_type = Extents;
  using index_type = typename extents_type::index_type;
  using size_type = typename extents_type::size_type;
  using rank_type = typename extents_type::rank_type;
  using layout_type = layout_right_padded<PaddingValue>;

private:
  static constexpr size_t rank_ = extents_type::rank(); // exposition only
  static constexpr size_t last-static-extent = // exposition only
    extents_type::static_extent(rank_ - 1);

  // [mdspan.layout.rightpadded.expo], exposition-only members
  static constexpr size_t static-padding-stride = /* see-below */; // exposition only

public:
  // [mdspan.layout.rightpadded.cons], constructors
  constexpr mapping() noexcept : mapping(extents_type{}) {}
  constexpr mapping(const mapping&) noexcept = default;
  constexpr mapping(const extents_type&);
  template<class OtherIndexType>
    constexpr mapping(const extents_type&, OtherIndexType);

  template<class OtherExtents>
    constexpr explicit(! is_convertible_v<OtherExtents, extents_type>)
      mapping(const layout_right::mapping<OtherExtents>&);
  template<class OtherExtents>
    constexpr explicit(rank_ > 0)
      mapping(const layout_stride::mapping<OtherExtents>&);
  template<class LayoutRightPaddedMapping>
    constexpr explicit( /* see below */ )
      mapping(const LayoutRightPaddedMapping&);
  template<class LayoutLeftPaddedMapping>
    constexpr explicit( /* see below */ )
      mapping(const LayoutLeftPaddedMapping&) noexcept;

  constexpr mapping& operator=(const mapping&) noexcept = default;

  // [mdspan.layout.rightpadded.obs], observers
  constexpr const extents_type& extents() const noexcept { return extents_; }
  constexpr array<index_type, rank_> strides() const noexcept;

  constexpr index_type required_span_size() const noexcept;

  template<class... Indices>
    constexpr index_type operator()(Indices...) const noexcept;

  static constexpr bool is_always_unique() noexcept { return true; }
  static constexpr bool is_always_exhaustive() noexcept;
  static constexpr bool is_always_strided() noexcept { return true; }

  static constexpr bool is_unique() noexcept { return true; }
  constexpr bool is_exhaustive() const noexcept;
  static constexpr bool is_strided() noexcept { return true; }

  constexpr index_type stride(rank_type) const noexcept;

  template<class LayoutRightPaddedMapping>
    friend constexpr bool operator==(
      const mapping&,
      const LayoutRightPaddedMapping&) noexcept;

private:
  // [mdspan.layout.rightpadded.expo], exposition-only members
  index_type stride-rm2 = static-padding-stride; // exposition only
  extents_type extents_{}; // exposition only

  // [mdspan.submdspan.mapping], submdspan mapping specialization
  template<class... SliceSpecifiers>
    constexpr auto submdspan-mapping-impl( // exposition only
      SliceSpecifiers...) const -> see below;
  template<class... SliceSpecifiers>
    friend constexpr auto submdspan_mapping(
      const mapping& src, SliceSpecifiers... slices) {
        return src.submdspan-mapping-impl(slices...);
    }
};
```

2 If `Extents` is not a specialization of `extents`, then the program is ill-formed.

3 `layout_right_padded::mapping<E>` is a trivially copyable type that models `regular` for each `E`.

4 Throughout [mdspan.layout.rightpadded], let `P_rank` be the following size *`rank_`* parameter pack of `size_t` values:

(4.1) — the empty parameter pack, if *`rank_`* equals zero; otherwise

(4.2) — `0zu`, if *`rank_`* equals one; otherwise

(4.3) — the parameter pack `0zu`, `1zu`, …, *`rank_`*`− 1`.

5 *Mandates:*

(5.1) — If `rank_dynamic() == 0` is `true`, then the size of the multidimensional index space `Extents()` is representable as a value of type `index_type`.

(5.2) — `padding_value` is representable as a value of type `index_type`.

(5.3) — If

  — *`rank_`* is greater than one,

  — `padding_value` does not equal `dynamic_extent`, and

  — *`last-static-extent`* does not equal `dynamic_extent`,

then *LEAST-MULTIPLE-AT-LEAST*(`padding_value` , *`last-static-extent`*) is representable as a value of type `size_t`, and is representable as a value of type `index_type`.

(5.4) — If

  — *`rank_`* is greater than one,

  — `padding_value` does not equal `dynamic_extent`, and

  — `extents_type::static_extent(k)` does not equal `dynamic_extent` for all `k` in the range [0, *`rank_`*),

then the product of *LEAST-MULTIPLE-AT-LEAST*(`padding_value` , `ext.static_extent(`*`rank_`*`− 1)`) and all values `ext.static_extent(k)` with `k` in the range of [0, *`rank_`* −1) is representable as a value of type `size_t`, and is representable as a value of type `index_type`.

§ 4.10.2 Exposition-only members [mdspan.layout.rightpadded.expo]

```
static constexpr size_t static-padding-stride = /* see-below */; // exposition only
```

1 The value is

(1.1) — `0`, if *`rank_`* equals zero or one; otherwise

(1.2) — `dynamic_extent`, if `padding_value` or *`last-static-extent`* equals `dynamic_extent`; otherwise

(1.3) — the `size_t` value which is *LEAST-MULTIPLE-AT-LEAST*(`padding_value` , *`last-static-extent`*).

```
index_type stride-rm2 = static-padding-stride; // exposition only
```

2 *Recommended practice*: Implementations should not store this value if *`static-padding-stride`* is not `dynamic_extent`. *[Note:* Using `extents<index_type,`*`static-padding-stride`*`>` instead of `index_type` as the type of *`stride-rm2`* would achieve this. *– end note]*

§ 4.10.3 Constructors [mdspan.layout.rightpadded.cons]

```
constexpr mapping(const extents_type& ext);
```

1 *Preconditions:*

(1.1) — The size of the multidimensional index space `ext` is representable as a value of type `index_type`.

(1.2) — If *`rank_`* is greater than one and `padding_value` does not equal `dynamic_extent`, then *LEAST-MULTIPLE-AT-LEAST*(`padding_value` , `ext.extent(`*`rank_`*`− 1)`) is representable as a value of type `index_type`.

(1.3) — If *`rank_`* is greater than one and `padding_value` does not equal `dynamic_extent`, then the product of *LEAST-MULTIPLE-AT-LEAST*(`padding_value` , `ext.extent(`*`rank_`*`− 1)`) and all values `ext.extent(k)` with `k` in the range of [0, *`rank_`*−1) is representable as a value of type `index_type`.

2 *Effects:*

(2.1) — Direct-non-list-initializes *`extents_`* with `ext`; and

(2.2) — if *`rank_`* is greater than one, direct-non-list-initializes *`stride-rm2`* with

  (2.2.1) — `ext.extent(`*`rank_`*`− 1)` if `padding_value` is `dynamic_extent`, otherwise with

  (2.2.2) — *LEAST-MULTIPLE-AT-LEAST*(`padding_value` , `ext.extent(`*`rank_`*`− 1)`).

```
template<class OtherIndexType>
constexpr mapping(const extents_type& ext, OtherIndexType pad);
```

3 *Constraints:*

(3.1) — `is_convertible_v<OtherIndexType, index_type>` is `true`.

(3.2) — `is_nothrow_constructible_v<index_type, OtherIndexType>` is `true`.

4 *Preconditions:*

(4.1) — `pad` is representable as a value of type `index_type`.

(4.2) — `extents_type::`*`index-cast`*`(pad)` is greater than zero.

(4.3) — If *`rank_`* is greater than one, then *LEAST-MULTIPLE-AT-LEAST*(`pad` , `ext.extent(`*`rank_`*`− 1)`) is representable as a value of type `index_type`.

(4.4) — If *`rank_`* is greater than one, then the product of *LEAST-MULTIPLE-AT-LEAST*(`pad` , `ext.extent(`*`rank_`*`− 1)`) and all values `ext.extent(k)` with `k` in the range of [0, *`rank_`*−1) is representable as a value of type `index_type`.

(4.5) — If `padding_value` is not equal to `dynamic_extent`, `padding_value` equals `extents_type::`*`index-cast`*`(pad)`.

5 *Effects:* Direct-non-list-initializes *`extents_`* with `ext`, and if *`rank_`* is greater than one, direct-non-list-initializes *`stride-rm2`* with *LEAST-MULTIPLE-AT-LEAST*(`pad` , `ext.extent(`*`rank_`*`− 1)`).

```
template<class OtherExtents>
  constexpr explicit(! is_convertible_v<OtherExtents, extents_type>)
    mapping(const layout_right::mapping<OtherExtents>& other);
```

6 *Constraints:* `is_constructible_v<extents_type, OtherExtents>` is `true`.

7 *Mandates:* If `OtherExtents::rank() > 1`, then `(`*`static-padding-stride`*`== dynamic_extent) || (OtherExtents::static_extent(`*`rank_`*`− 1) == dynamic_extent) || (`*`static-padding-stride`*`== OtherExtents::static_extent(`*`rank_`*`− 1))` is `true`.

8 *Preconditions:*

(8.1) — If *`rank_`*` > 1` is `true` and `padding_value == dynamic_extent` is `false`, then `other.stride(`*`rank_`*`− 2)` equals *LEAST-MULTIPLE-AT-LEAST*(`padding_value` , `extents_type::`*`index-cast`*`(other.extents().extent(`*`rank_`*`− 1))`); and

(8.2) — `other.required_span_size()` is representable as a value of type `index_type`.

9 *Effects:* Equivalent to `mapping(other.extents())`.

```
template<class OtherExtents>
  constexpr explicit(rank_ > 0)
    mapping(const layout_stride::mapping<OtherExtents>& other);
```

10 *Constraints:* `is_constructible_v<extents_type, OtherExtents>` is `true`.

11 *Preconditions:*

(11.1) — If *`rank_`* is greater than 1 and `padding_value` does not equal `dynamic_extent`, then `other.stride(`*`rank_`*`− 2)` equals *LEAST-MULTIPLE-AT-LEAST*(`padding_value` , `extents_type::`*`index-cast`*`(other.extents().extent(`*`rank_`*`− 1))`).

(11.2) — If *`rank_`* is greater than 0, then `other.stride(`*`rank_`*`− 1)` equals 1.

(11.3) — If *`rank_`* is greater than 2, then for all `r` in the range [`0`, *`rank_`*`− 2`), `other.stride(r)` equals `(other.extents().`*`rev-prod-of-extents`*`(r) / other.extents().extent(`*`rank_`*`− 1)) * other.stride(`*`rank_`*`− 2)`.

(11.4) — `other.required_span_size()` is representable as a value of type `index_type`.

12 *Effects:*

(12.1) — Direct-non-list-initializes *`extents_`* with `other.extents()`; and

(12.2) — if *`rank_`* is greater than one, direct-non-list-initializes *`stride-rm2`* with `other.stride(`*`rank_`*`− 2)`.

```
template<class LayoutRightPaddedMapping>
  constexpr explicit( /* see below */ )
    mapping(const LayoutRightPaddedMapping& other);
```

13 *Constraints:*

(13.1) — *`is-layout-right-padded-mapping-of`*`<LayoutRightPaddedMapping>` is `true`.

(13.2) — `is_constructible_v<extents_type, typename LayoutRightPaddedMapping::extents_type>` is `true`.

14 *Mandates:* If *`rank_`* is greater than 1, then `padding_value == dynamic_extent || LayoutRightPaddedMapping::padding_value == dynamic_extent || padding_value == LayoutRightPaddedMapping::padding_value` is `true`.

15 *Preconditions:*

(15.1) — If *`rank_`* is greater than 1 and `padding_value` does not equal `dynamic_extent`, then `other.stride(`*`rank_`*`− 2)` equals *LEAST-MULTIPLE-AT-LEAST*(`padding_value` , `extents_type::`*`index-cast`*`(other.extent(`*`rank_`*`− 1))`).

(15.2) — `other.required_span_size()` is representable as a value of type `index_type`.

16 *Effects:*

(16.1) — Direct-non-list-initializes *`extents_`* with `other.extents()`; and

(16.2) — if *`rank_`* is greater than one, direct-non-list-initializes *`stride-rm2`* with `other.stride(`*`rank_`*`− 2)`.

17 *Remarks:* The expression inside `explicit` is equivalent to: *`rank_`*`> 1 && (padding_value != dynamic_extent || LayoutRightPaddedMapping::padding_value == dynamic_extent)`.

```
template<class LayoutLeftPaddedMapping>
  constexpr explicit( /* see below */ )
    mapping(const LayoutLeftPaddedMapping& other) noexcept;
```

18 *Constraints:*

(18.1) — *`is-layout-left-padded-mapping-of`*`<LayoutLeftPaddedMapping>` is `true` or *`is-mapping-of`*`<layout_left, LayoutLeftPaddedMapping>` is `true`.

(18.2) — *`rank_`* equals zero or one.

(18.3) — `is_constructible_v<extents_type, typename LayoutLeftPaddedMapping::extents_type>` is `true`.

19 *Preconditions:* `other.required_span_size()` is representable as a value of type `index_type`.

20 *Effects:* Direct-non-list-initializes *`extents_`* with `other.extents()`.

21 *Remarks:* The expression inside `explicit` is equivalent to: `!is_convertible_v<typename LayoutLeftPaddedMapping::extents_type, extents_type>`.

[Note: Neither the input mapping nor the mapping to be constructed uses the padding stride in the rank-0 or rank-1 case, so the padding stride affects neither the constraints nor the preconditions. *– end note]*

§ 4.10.4 Observers [mdspan.layout.rightpadded.obs]

```
constexpr array<index_type, rank_> strides() const noexcept;
```

1 *Returns:* `array<index_type,`*`rank_`*`>({stride(P_rank)...})`.

```
constexpr index_type required_span_size() const noexcept;
```

2 *Returns:* `0` if the multidimensional index space *`extents_`* is empty, otherwise `*this(((extents_(P_rank) - index_type(1))...)) + 1`.

```
template<class... Indices>
constexpr size_t operator()(Indices... idxs) const noexcept;
```

3 *Constraints:*

(3.1) — `sizeof...(Indices) ==`*`rank_`* is `true`.

(3.2) — `(is_convertible_v<Indices, index_type> && ...)` is `true`.

(3.3) — `(is_nothrow_constructible_v<index_type, Indices> && ...)` is `true`.

4 *Precondition:* `extents_type::`*`index-cast`*`(idxs)` is a multidimensional index in `extents()` (*[mdspan.overview]*).

5 *Returns:* `((static_cast<index_type>(idxs) * stride(P_rank)) + ... + 0)`.

```
static constexpr bool is_always_exhaustive() noexcept;
```

6 *Returns:*

(6.1) — If *`rank_`* equals zero or one, then `true`;

(6.2) — else, if neither *`static-padding-stride`* nor *`last-static-extent`* equal `dynamic_extent`, then *`static-padding-stride`*`==`*`last-static-extent`*;

(6.3) — otherwise, `false`.

```
constexpr bool is_exhaustive() const noexcept;
```

7 *Returns:* `true` if *`rank_`* equals zero or one; otherwise, *`extents_`*`.extent(`*`rank_`*`− 1) == stride(`*`rank_`*`− 2)`.

```
constexpr index_type stride(rank_type r) const noexcept;
```

8 *Preconditions:* `r` is smaller than *`rank_`*.

9 *Returns:*

(9.1) — If `r` equals *`rank_`*`− 1`: `1`;

(9.2) — else, if `r` equals *`rank_`*`− 2`: *`stride-rm2`*;

(9.3) — otherwise, the product of *`stride-rm2`* and all values *`extents_`*`.extent(k)` with `k` in the range of [`r + 1`, *`rank_`*`− 1`).

```
template<class LayoutRightPaddedMapping>
  friend constexpr bool operator==(
    const mapping& x,
    const LayoutRightPaddedMapping& y) noexcept;
```

10 *Constraints:*

(10.1) — *`is-layout-right-padded-mapping-of`*`<LayoutRightPaddedMapping>` is `true`.

(10.2) — `LayoutRightPaddedMapping::extents_type::rank() ==`*`rank_`* is `true`.

11 *Returns:* `true` if `x.extents() == y.extents()` is `true` and *`rank_`*` < 2 || x.stride(`*`rank_`*`− 2) == y.stride(`*`rank_`*`− 2)` is `true`. Otherwise, `false`.

§ 4.11 Layout specializations of `submdspan_mapping` [mdspan.submdspan.mapping]

Replace Section � [mdspan.submdspan.mapping] (“Layout specializations of `submdspan_mapping`”), with:

24.7.3.7.6 Specialization of `submdspan_mapping` [mdspan.submdspan.mapping]

24.7.3.7.6.1 Common [mdspan.submdspan.mapping.common]

1 The following elements apply to all functions in [mdspan.submdspan.mapping].

2 *Constraints:* `sizeof...(slices)` equals `extents_type::rank()`.

3 *Mandates:* For each rank index `k` of `extents()`, exactly one of the following is true:

(3.1) — S_k models `convertible_to<index_type>`,

(3.2) — S_k models *`index-pair-like`*`<index_type>`,

(3.3) — `is_convertible_v<`S_k`, full_extent_t>` is `true`, or

(3.4) — S_k is a specialization of `strided_slice`.

4 *Preconditions:* For each rank index `k` of `extents()`, all of the following are true:

(4.1) — if S_k is a specialization of `strided_slice`,

  — s_k`.extent` $= 0$ or

  — s_k`.stride` > 0; and

(4.2) — 0≤ `first_<index_type, k>(slices...)` ≤ `last_<k>(extents(), slices...)` ≤ `extents.extent(k)`. *[Editorial Note:* There is a drive-by fix here, where the current draft text just doesnt make sense. *- end note]*

5 Let `sub_ext` be the result of `submdspan_extents(extents(), slices...)` and let `SubExtents` be `decltype(sub_ext)`.

6 Let `sub_strides` be an `array<SubExtents::index_type, SubExtents::rank()>` such that for each rank index `k` of `extents()` for which *`map-rank`*`[k]` is not `dynamic_extent`, `sub_strides[`*`map-rank`*`[k]]` equals:

(6.1) — `stride(k) *`*`de-ice`*`(`s_k`.stride)` if S_k is a specialization of `strided_slice` and s_k`.stride` $<$ s_k`.extent`;

(6.2) — otherwise, `stride(k)`.

7 Let `P` be a parameter pack such that `is_same_v<make_index_sequence<rank()>, index_sequence<P...>>` is `true`.

8 Let `offset` be a value of type `size_t` equal to `(*this)(first_<index_type, P>(slices...)...)`.

24.7.3.7.6.2 `layout_left` specialization of `submdspan_mapping` [mdspan.submdspan.mapping.left]

```
template<class Extents>
template<class... SliceSpecifiers>
constexpr auto layout_left::mapping<Extents>::submdspan-mapping-impl(    // exposition only
  SliceSpecifiers... slices) const -> see below;
```

1 *Returns:*

(1.1) — `submdspan_mapping_result{*this, 0}`, if *`rank_`*`==0` is `true`;

(1.2) — otherwise, `submdspan_mapping_result{layout_left::mapping(sub_ext), offset}`, if `SubExtents::rank()==0` is `true`;

(1.3) — otherwise, `submdspan_mapping_result{layout_left::mapping(sub_ext), offset}`, if

  — for each `k` in the range [0, `SubExtents::rank()-1`), `is_convertible_v<` S_k `, full_extent_t>` is `true`; and

  — for `k` equal to `SubExtents::rank()-1`, S_k models *`index-pair-like`*`<index_type>` or `is_convertible_v<`S_k`, full_extent_t>` is `true`;

  [Note: If the above conditions are true, all S_k with `k` larger than `SubExtents::rank()-1` are convertible to `index_type`. *- end note]*

(1.4) — otherwise, `submdspan_mapping_result{layout_left_padded<S_static>::mapping(sub_ext, stride(u+1)), offset}`, if for a value `u` for which `u+1` is the smallest value `p` larger than zero for which S_p models *`index-pair-like`*`<index_type>` or `is_convertible_v<`S_p`, full_extent_t>` is `true`, the following conditions are met:

  — S_0 models *`index-pair-like`*`<index_type>` or `is_convertible_v<` S_0 `, full_extent_t>` is `true`; and

  — for each `k` in the range [`u + 1`, `u + SubExtents::rank() - 1`), `is_convertible_v<`S_k`, full_extent_t>` is `true`; and

  — for `k` equal to `u + SubExtents::rank()-1`, S_k models *`index-pair-like`*`<index_type>` or `is_convertible_v<`S_k`, full_extent_t>` is `true`;

  and where `S_static` is:

  — `dynamic_extent`, if `static_extent(k)` is `dynamic_extent` for any `k` in the range [`0`, `u + 1`), otherwise

  — the product of all values `static_extent(k)` for `k` in the range [`0`, `u + 1`);

(1.5) — otherwise, `submdspan_mapping_result{layout_stride::mapping(sub_ext, sub_strides), offset}`.

24.7.3.7.6.3 `layout_right` specialization of `submdspan_mapping` [mdspan.submdspan.mapping.right]

```
template<class Extents>
template<class... SliceSpecifiers>
constexpr auto layout_right::mapping<Extents>::submdspan-mapping-impl(   // exposition only
  SliceSpecifiers... slices) const -> see below;
```

1 *Returns:*

(1.1) — `submdspan_mapping_result{*this, 0}`, if *`rank_`*`==0` is `true`;

(1.2) — otherwise, `submdspan_mapping_result{layout_right::mapping(sub_ext), offset}`, if `SubExtents::rank()==0` is `true`;

(1.3) — otherwise, `submdspan_mapping_result{layout_right::mapping(sub_ext), offset}`, if

  — for each `k` in the range [*`rank_`*`− SubExtents::rank()+1`, *`rank_`*), `is_convertible_v<` S_k `, full_extent_t>` is `true`; and

  — for `k` equal to *`rank_`*`− SubExtents::rank()`, S_k models *`index-pair-like`*`<index_type>` or `is_convertible_v<`S_k`, full_extent_t>` is `true`;

  [Note: If the above conditions are true, all S_k with `k` < *`rank_`*`− SubExtents::rank()` are convertible to `index_type`. *- end note]*

(1.4) — otherwise, `submdspan_mapping_result{layout_right_padded<S_static>::template mapping(sub_ext, stride(`*`rank_`*`−u−2)), offset}` if for a value `u` for which *`rank_`*`− u − 2` is the largest value `p` smaller than *`rank_`*`− 1` for which S_p models *`index-pair-like`*`<index_type>` or `is_convertible_v<`S_p`, full_extent_t>` is `true`, the following conditions are met:

  — for `k` equal to *`rank_`*`− 1`, S_k models *`index-pair-like`*`<index_type>` or `is_convertible_v<` S_k `, full_extent_t>` is `true`; and

  — for each `k` in the range [*`rank_`*`− SubExtents::rank() − u + 1`, *`rank_`*`− u − 1`), `is_convertible_v<`S_k`, full_extent_t>` is `true`; and

  — for `k` equal to *`rank`*`− SubExtents::rank() − u`, S_k models *`index-pair-like`*`<index_type>` or `is_convertible_v<`S_k`, full_extent_t>` is `true`;

  and where `S_static` is:

  — `dynamic_extent` if for any `k` in the range [*`rank_`*`− u − 1`, *`rank_`*) `static_extent(k)` is `dynamic_extent`, otherwise

  — the product of all values `static_extent(k)` with `k` in the range [*`rank_`*`− u − 1`, *`rank_`*);

(1.5) — otherwise, `submdspan_mapping_result{layout_stride::mapping(sub_ext, sub_strides), offset}`.

24.7.3.7.6.4 `layout_stride` specialization of `submdspan_mapping` [mdspan.submdspan.mapping.stride]

```
template<class Extents>
template<class... SliceSpecifiers>
constexpr auto layout_stride::mapping<Extents>::submdspan-mapping-impl(  // exposition only
  SliceSpecifiers ... slices) const -> see below;
```

1 *Returns:*

(1.1) — `submdspan_mapping_result{*this, 0}`, if `Extents::rank()==0` is `true`;

(1.2) — otherwise, `submdspan_mapping_result{layout_stride::mapping(sub_ext, sub_strides), offset}`.

24.7.3.7.6.5 `layout_left_padded` specialization of `submdspan_mapping` [mdspan.submdspan.mapping.leftpadded]

```
template<class Extents>
template<class... SliceSpecifiers>
constexpr auto layout_left_padded::mapping<Extents>::submdspan-mapping-impl(    // exposition only
  SliceSpecifiers... slices) const -> see below;
```

1 *Returns:*

(1.1) — `submdspan_mapping_result{*this, 0}`, if `Extents::rank()==0` is `true`;

(1.2) — otherwise, `submdspan_mapping_result{layout_left::mapping(sub_ext), offset}`, if

  — *`rank_`*`==1` is `true`; or

  — `SubExtents::rank()==0` is `true`;

(1.3) — otherwise, `submdspan_mapping_result{layout_left::mapping(sub_ext), offset}`, if

  — `SubExtents::rank()==1` is `true`; and

  — S_0 models *`index-pair-like`*`<index_type>` or `is_convertible_v<` S_0 `, full_extent_t>` is `true`;

(1.4) — otherwise, `submdspan_mapping_result{layout_left_padded<S_static>::mapping(sub_ext, stride(u + 1)), offset}`, if for a value `u` for which `u + 1` is the smallest value `p` larger than zero for which S_p models *`index-pair-like`*`<index_type>` or `is_convertible_v<`S_p`, full_extent_t>` is `true`, the following conditions are met:

  — S_0 models *`index-pair-like`*`<index_type>` or `is_convertible_v<`S_0`, full_extent_t>` is `true`; and

  — for each `k` in the range [`u + 1`, `u + SubExtents::rank()-1`), `is_convertible_v<`S_k`, full_extent_t>` is `true`; and

  — for `k` equal to `u + SubExtents::rank()-1`, S_k models *`index-pair-like`*`<index_type>` or `is_convertible_v<`S_k`, full_extent_t>` is `true`;

  and where `S_static` is:

  — `dynamic_extent`, if *`static-padding-stride`* is `dynamic_extent` or `static_extent(k)` is `dynamic_extent` for any `k` in the range [`1`, `u + 1`), otherwise

  — the product of *`static-padding-stride`* and all values `static_extent(k)` for `k` in the range [`1`, `u + 1`);

(1.5) — otherwise, `submdspan_mapping_result{layout_stride::mapping(sub_ext, sub_strides), offset}`.

24.7.3.7.6.6 `layout_right_padded` specialization of `submdspan_mapping` [mdspan.submdspan.mapping.rightpadded]

```
template<class Extents>
template<class... SliceSpecifiers>
constexpr auto layout_right_padded::mapping<Extents>::submdspan-mapping-impl(    // exposition only
  SliceSpecifiers... slices) const -> see below;
```

1 *Returns:*

(1.1) — `submdspan_mapping_result{*this, 0}`, if *`rank_`*`== 0` is `true`;

(1.2) — otherwise, `submdspan_mapping_result{layout_right::mapping(sub_ext), offset}`, if

  — *`rank_`*`==1` is `true`; or

  — `SubExtents::rank()==0` is `true`;

(1.3) — otherwise, `submdspan_mapping_result{layout_right::mapping(sub_ext), offset}`, if

  — `SubExtents::rank()==1` is `true`; and

  — for `k` equal to *`rank_`*`− 1`, S_k models *`index-pair-like`*`<index_type>` or `is_convertible_v<` S_k `, full_extent_t>` is `true`;

(1.4) — otherwise, `submdspan_mapping_result{layout_right_padded<S_static>::template mapping(sub_ext, stride(`*`rank_`*`−u−2)), offset}` if for a value `u` for which *`rank_`*`− u − 2` is the largest value `p` smaller than *`rank_`*`− 1` for which S_p models *`index-pair-like`*`<index_type>` or `is_convertible_v<`S_p`, full_extent_t>` is `true`, the following conditions are met:

  — for `k` equal to *`rank_`*`− 1`, S_k models *`index-pair-like`*`<index_type>` or `is_convertible_v<` S_k `, full_extent_t>` is `true`; and

  — for each `k` in the range [*`rank_`*`− SubExtents::rank() − u + 1`, *`rank_`*`− u − 1`), `is_convertible_v<`S_k`, full_extent_t>` is `true`; and

  — for `k` equal to *`rank_`*`− SubExtents::rank() − u`, S_k models *`index-pair-like`*`<index_type>` or `is_convertible_v<`S_k`, full_extent_t>` is `true`;

  and where `S_static` is:

  — `dynamic_extent` if *`static-padding-stride`* is `dynamic_extent` or for any `k` in the range [*`rank_`*`− u − 1`, *`rank_`*`− 1`) `static_extent(k)` is `dynamic_extent`, otherwise

  — the product of *`static-padding-stride`* and all values `static_extent(k)` with `k` in the range [*`rank_`*`− u − 1`, *`rank_`*`− 1`);

(1.5) — otherwise, `submdspan_mapping_result{layout_stride::mapping(sub_ext, sub_strides), offset}`.